

Backup & Restore Runbook

Lesotho Farmers Portal — operational procedures

Lesotho Farmers Portal — implementation team

2026-05-06

Table of Contents

Backup & Restore Runbook — Lesotho Farmers Portal

Audience: MAFSN ICT operations. **Purpose:** Step-by-step procedures for backing up the system and restoring from any of four failure scenarios. Includes RTO/RPO targets and a quarterly test schedule. **Pre-condition:** Read the MAFSN ICT TO-DO (section 4d) first — this runbook elaborates on what's there.

The system is a Joget DX 8.1 application with state in four places. Each is backed up differently and restored differently. This runbook walks each.

What needs backing up

Component	Where it lives	How it's backed up	Why it matters
Postgres database	Azure-managed Postgres (joget-pgsql-sa)	Azure-managed snapshot	All citizen + transaction data
Joget application files	wflow/app_plugins/*.jar, the JWA	Git repository (gs-plugins/, _forms/, etc.)	The application code + form definitions
File uploads	VM filesystem wflow/app_formup loads/	Rsync to a separate Azure Storage account	Signatures, invoice scans (PII)
Joget config	wflow/wflow-postgres/META-INF/ wflow.properties + Tomcat config	Manual snapshot to git	Connection pool, JVM tuning

Recovery point objective (RPO): 24 hours — i.e., we can lose at most one day's data in the worst case. Set by the daily Azure snapshot frequency.

Recovery time objective (RTO): 4 hours from incident declaration to service restoration in the worst (full-VM-loss) case. Smaller incidents restore in under an hour.

Component 1 — Postgres backup + restore

Backup procedure

Azure Postgres handles this automatically once configured. To verify:

From an Azure CLI session

```
az postgres flexible-server backup list \
  --resource-group <rg-name> \
  --name joget-pgsql-sa
```

Expected output: at least 30 snapshots, one per day, retention 30 days.

If the list is empty or the most recent snapshot is older than 24 hours, **escalate immediately to the Azure admin**. Verify in the Azure portal:

1. Open the joget-pgsql-sa resource.
2. Go to **Backup and restore**.
3. Confirm: backup retention = 30 days, geo-redundant backup = enabled.

Restore procedure (point-in-time)

For most scenarios you'll restore to a *new* Postgres instance, then swap the connection string. Don't restore in-place over the live DB; you may overwrite good data.

1. **Identify the restore target time.** What's the latest known-good moment? E.g., "yesterday 03:00 UTC, before the bad import script ran". Note this in your incident log.
2. **Trigger the restore in Azure portal:**
 - Open joget-pgsql-sa → **Backup and restore** → **Restore**.
 - Pick the target time.
 - Choose a new server name: joget-pgsql-sa-restore-<YYYYMMDDHHmm>.
 - Same SKU, same region, same VNet.
 - Submit. Wait 15–30 minutes for the restore to complete.
3. **Verify the restored DB:**

```
psql -h <restored-host> -U jogetadmin -d jogetdb \
  -c "SELECT count(*) FROM app_fd_im_voucher;"
```

Sanity-check counts on a few key tables. Cross-reference with the incident timeline.

4. **Switch the application to the restored DB:**

- SSH to the application VM.
- Edit wflow/wflow-postgres/META-INF/wflow.properties:

```
wflowDataSource.url=jdbc:postgresql://<restored-host>:5432/
jogetdb?...
```

- Restart Tomcat: `sudo systemctl restart joget`.
- Wait ~60 seconds for the app to come up.

5. **Sanity-check the running app:**

- Open the userview in a browser.
- Open **Reports** → **IM Dashboard KPIs**: numbers should match what you saw at the restore-target time.
- Run `_tooling/test_im_e2e.py` against the restored environment; all assertions should pass.

6. **Communicate to operators:** “System is back up at . Any activity between and may need to be re-entered.”

7. **Post-incident:** retain the failed instance (don’t delete) until the incident review is closed. Document root cause + corrective action in `_design/incident_log.md`.

Estimated time: 30 min to provision + 5 min to swap config + 10 min to verify = **45 min total** for a Postgres-only incident.

Component 2 — Plugin JARs

Backup procedure

The plugin source lives in the repo at `gs-plugins/<plugin>/`. Built artefacts (the .jar files) are reproducible from source.

Discipline:

1. **Every JAR-deploy must be preceded by a git commit** of the sources. The build-NNN counter in `Build.java` is the audit trail.
2. **Tag releases.** When a build goes to production, tag it:

```
git tag -a build-109 -m "MDM list endpoint, smoke-tested 2026-05-06"
git push origin build-109
```

3. **Store the built JARs alongside the tag.** Either GitHub Releases (free for public repos), or an Azure Storage container with versioned blob names like `reg-bb-engine-build-109.jar`.

Restore procedure

If a plugin breaks in production:

1. **Identify the last known-good build.**
 - Check `_design/decision-log.md` for the latest stamped build.
 - Or query the live DB:

```
SELECT json FROM app_form
WHERE appid='farmersPortal' AND formid='im_voucher';
```

and grep the result for build numbers in plugin classNames.

- Cross-reference with the joget.log startup line:

```
reg-bb-engine starting — build-NNN @ <timestamp>
```

2. Pull the last-good JAR:

- From git tag: git checkout build-NNN; cd gs-plugins/<plugin>; bash deploy/repack.sh
- From Azure Storage: download the blob.

3. Upload to Joget:

- App Composer → Settings → Manage Plugins → upload the JAR.
- Confirm in joget.log that the new build's Activator log line appears.
- Smoke-test by triggering one operation that uses the plugin (e.g. issue a voucher, run the e2e test).

Estimated time: 15 min total.

Component 3 — File uploads (wflow/app_formuploads/)

This is where signature PNGs and invoice scans live. The Postgres database stores only the filenames; the actual files are on the VM.

Backup procedure

Set up a daily rsync to a separate Azure Storage account (do this once, then it runs automatically):

```
# /etc/cron.d/joget-file-backup
0 2 * * * jogetadmin rsync -avz --delete \
/opt/joget/wflow/app_formuploads/ \
/mnt/azure-backup/app_formuploads/ \
>> /var/log/joget-file-backup.log 2>&1
```

/mnt/azure-backup/ is an Azure Files share mounted on the VM. Configure it once via Azure Portal → Storage Account → File Share.

Verify weekly:

```
# On the VM
ls -la /mnt/azure-backup/app_formuploads/im_distribution/ | head -5
# Should show recent receipt directories with farmer_signature.png
```

Restore procedure

If a file is missing or corrupted:

1. **Identify the missing file.** The DB has the filename; trace from there:

```
SELECT id, c_voucher_code, c_farmer_signature
FROM app_fd_im_distribution
WHERE id = '<receipt-id>';
```

2. **Find the file in the backup share:**

```
/mnt/azure-backup/app_formuploads/im_distribution/<receipt-id>/
farmer_signature.png
```

3. **Copy back to live:**

```
sudo cp /mnt/azure-backup/app_formuploads/im_distribution/<id>/* \
/opt/joget/wflow/app_formuploads/im_distribution/<id>/
sudo chown joget:joget
/opt/joget/wflow/app_formuploads/im_distribution/<id>/*
```

4. **Verify in the UI:** open the receipt, confirm the signature renders.

Estimated time: 5 min per file restored.

Component 4 — Joget application config

Tomcat config (server.xml, setenv.sh), the wflow.properties file, and any environment-specific overrides should be snapshot to git whenever changed. This is rare but matters during full-VM rebuild.

Backup procedure

1. **Make a one-time snapshot:**

```
mkdir -p ~/joget-config-backup
cp /opt/joget/wflow/wflow-postgres/META-INF/wflow.properties ~/joget-config-backup/
cp /opt/joget/apache-tomcat-*/conf/server.xml ~/joget-config-backup/
cp /opt/joget/apache-tomcat-*/bin/setenv.sh ~/joget-config-backup/
cp /etc/systemd/system/joget.service ~/joget-config-backup/
```

2. **Commit to a private git repo** (separate from the application source repo, since these may contain secrets).
3. **After every config change**, repeat steps 1 and 2 with a commit message describing what changed and why.

Restore procedure

When rebuilding the VM:

1. Install Joget DX 8.1 from the customer's installer.
2. Stop the service.
3. Replace each file from the config-backup repo.
4. Restart the service.

5. Verify the application loads + connects to the DB.

Estimated time: 10 min if done properly, an hour if the config repo has drifted.

Disaster scenarios

Four scenarios, in increasing severity:

Scenario A — Single bad import (Postgres data corruption only)

Symptoms: bad data visible in datalists, but the system is otherwise running.

1. Postgres restore (Component 1, point-in-time to before the import)
2. Communicate to operators
3. Re-do any legitimate work that fell in the lost window
4. Update _design/incident_log.md

RTO: 1 hour

Scenario B — Plugin regression breaks the system

Symptoms: App Composer shows “plugins not installed” errors; certain forms refuse to render or save.

1. Plugin JAR rollback (Component 2, last known-good build)
2. Smoke-test the affected screens
3. Investigate the regression in dev; do NOT re-deploy until verified

RTO: 30 minutes

Scenario C — File store partial loss

Symptoms: signature PNGs or invoice scans missing on receipts / stock-transactions; UI shows broken-image icon.

1. File restore from Azure Files backup (Component 3)
2. Spot-check a sample of restored files

RTO: 30 minutes for a known set, longer if you have to scan for all damage

Scenario D — Full VM loss (catastrophic)

Symptoms: VM unreachable; could be Azure-region-wide outage, security incident, or accidental delete.

1. Provision a new VM in a different availability zone (or different region if the outage is regional).
2. Install Joget DX 8.1 base.
3. Restore Joget config (Component 4) — wflow.properties, setenv.sh, server.xml, systemd unit.

4. Deploy the latest known-good plugin JARs (Component 2) — fastest from Azure Storage if mirrored there.
5. Trigger Postgres restore to a fresh instance (Component 1) and point the new VM's wflow.properties at it.
6. Restore file uploads (Component 3) — rsync from the Azure Files backup to the new VM's wflow/app_formuploads/.
7. Update DNS to point at the new VM's IP.
8. Smoke-test end-to-end: login, e2e test, sample ops.
9. Communicate “system back online at ” to all stakeholders.

RTO: 4 hours

This is the worst-case scenario and the one that justifies the quarterly test cycle below.

Quarterly test cycle

A backup that has never been restored is not a backup. Test quarterly:

Quarter	What to test	Pass criteria
Q1	Postgres point-in-time restore to a non-prod instance	DB restores; counts match a known checkpoint
Q2	Plugin rollback on dev environment	Old JAR loads, smoke test passes
Q3	File-store rsync restore (one receipt's signature)	Signature renders correctly in UI
Q4	Full DR drill (Scenario D, simulated)	Service restored to a parallel VM in under 4 hours

Each test produces a one-page report filed under `_design/dr_test_reports/<YYYY-Q[1-4]>.md`. If any test fails, the finding goes on the incident log and the runbook is updated.

Backup verification daily check

Even with Azure-managed everything, run a simple verification daily (can be an automated script):

```
#!/bin/bash
# /opt/joget/scripts/check-backups.sh
# Run via cron at 03:00 UTC daily, after the snapshot window closes.
```

set -e

```
# Postgres snapshot age
LAST_SNAPSHOT=$(az postgres flexible-server backup list \
  --resource-group <rg> --name joget-pgsql-sa \
```

```

--query 'reverse(sort_by([], &backupName))[0].completedTime' -o tsv)
NOW=$(date -u +%s)
THEN=$(date -u -d "$LAST_SNAPSHOT" +%s)
AGE_HOURS=$(( ( NOW - THEN ) / 3600 ))

if [ $AGE_HOURS -gt 26 ]; then
    echo "ALERT: Postgres last snapshot is $AGE_HOURS hours old (expected ≤
24+2 grace)"
    # Send to your ops channel — Teams webhook, email, whatever.
fi

# File-store backup age
LAST_RSYNC=$(stat -c '%Y' /mnt/azure-backup/.rsync-stamp)
RSYNC_AGE_HOURS=$(( ( NOW - LAST_RSYNC ) / 3600 ))

if [ $RSYNC_AGE_HOURS -gt 26 ]; then
    echo "ALERT: File-store rsync last ran $RSYNC_AGE_HOURS hours ago"
fi

# Plugin tags up to date in git
LAST_TAG=$(git -C /opt/joget/lst-frm-prj tag -l 'build-*' | sort -V | tail -1)
LAST_JAR=$(ls -t /opt/joget/wflow/app_plugins/reg-bb-engine-*.jar | head -1)
JAR_BUILD=$(unzip -p "$LAST_JAR" 'global/govstack/regbb/engine/Build.class'
2>/dev/null \
| strings | grep -oE 'build-[0-9]+' | head -1)

if [ "$LAST_TAG" != "$JAR_BUILD" ]; then
    echo "ALERT: Latest deployed JAR ($JAR_BUILD) is not git-tagged ($LAST_TAG
is the latest tag)"
fi

echo "Backup checks passed at $(date -u)"

```

The cron schedule:

```

# /etc/cron.d/joget-backup-checks
0 3 * * * jogetadmin /opt/joget/scripts/check-backups.sh \
>> /var/log/joget-backup-checks.log 2>&1

```

If any check alerts, the on-call operator investigates within 1 hour.

Restore exercise log

Keep a log of every restore exercise (real incident or quarterly test):

Date	Type	Component	Outcome	RTO actual	Notes
2026-Q1	Test	Postgres PITR	Passed	32 min	Used staging
2026-Q2	Test	Plugin rollback	Passed	12 min	reg-bb-engine
109→108					

2026-Q3	Test	File-store restore	Passed	4 min	One signature
2026-Q4	Test	Full DR drill	Passed	3h 41m	Within RTO
2027-01-15	Real	Postgres data fix	Passed	47 min	Bad WFP import; restored to 03:00

The test log is not optional — donor audits ask for it.

Roles + escalation

Role	Owns
MAFSN ICT — Joget admin	Component 2, 3, 4 (JARs, files, config)
MAFSN ICT — Azure admin	Component 1 (Postgres) and the underlying VM
MAFSN ICT — security lead	Coordinates incident response across both above
Implementation team (us)	Documenting fixes for code-side bugs surfaced by the incident

Escalation path for an incident:

1. Operator reports → Joget admin investigates (15 min)
2. Joget admin can't recover → Azure admin called in (15 min)
3. Both can't recover within 1 hour → security lead declares incident
 - customer notification (depends on PII scope)
4. Implementation team called in for any bug fix needed — same day if high-severity, next business day for medium

Final checklist before opening to production

Before MAFSN exposes the system to real citizens:

- ☐ Component 1 — Azure Postgres backup retention confirmed at 30 days, geo-redundant enabled
- ☐ Component 2 — Latest plugin JARs git-tagged + mirrored to Azure Storage
- ☐ Component 3 — Daily rsync cron is running, last successful run is < 24h old
- ☐ Component 4 — Joget config files committed to private repo
- ☐ Daily backup-verification cron is running and reporting to ops
- ☐ At least one quarterly test cycle has completed successfully
- ☐ Incident log + restore exercise log are filed and reviewed
- ☐ Roles + escalation path are documented and the on-call rota is set

Until every box is ticked, the system isn't production-ready regardless of feature completeness.

Authored: 2026-05-06 by the implementation team. Revise after each quarterly test cycle to reflect what changed.